Envoy Upstream Queue Policy Extensions: Stochastic Fair Queuing (SFQ), Architecture, and Benchmarks

Technical Reference & Presentation Whitepaper

October 2026

Executive Summary

When an Envoy upstream connection pool reaches its concurrency or connection limit, incoming requests wait in an in-memory pending stream queue (`ConnPoolImplBase::pending_stream_queue_`). Historically, Envoy hardcoded this queue as a single First-In, First-Out (FIFO) list. Under FIFO, a single high-rate tenant bursting above upstream capacity fills the shared queue and inflicts severe Head-of-Line (HoL) blocking on every other tenant sharing the cluster.

To solve this, Envoy introduced the `envoy.queue_policy` extension point (`Cluster.queuing_policies.pending_rq_policy`) and the **Fair Queue Policy** (`envoy.queue_policy.fair_queue`). The fair queue policy implements **Stochastic Fair Queuing (SFQ)** over dynamic request metadata in $O(1)$ time and bounded memory. In multi-tenant Nighthawk benchmarks under a 3.8× noisy-neighbor burst, replacing FIFO with SFQ reduces well-behaved client **P99 latency by 14.6**× (from ∼ **680** ms to **44**–**49** ms), reduces mean latency by **17**× (from ∼ **339** ms to **18**–**21** ms), and cuts total Envoy pending queue depth by **51%** while remaining work-conserving.

Contents

1 Problem Statement: Head-of-Line Blocking in Upstream Connection Pools **2**
- 1.1 When Requests Enter the Pending Stream Queue . . . 2
- 1.2 The Multi-Tenant Noisy-Neighbor Failure Mode . . . 2

2 Architecture of the `envoy.queue_policy` Extension Point **3**
- 2.1 Cluster Proto Configuration and Load-Time Resolution . . . 3
- 2.2 Core Interfaces: `QueueBase` and `QueueItemMetadata` . . . 4
- 2.3 Dual-Container Ownership Model in `ConnPoolImplBase` . . . 4

3 End-to-End Request Classification Pipeline **6**

4 The Fair Queue Policy (`envoy.queue_policy.fair_queue`) & SFQ Data Structure **7**
- 4.1 Why Stochastic Fair Queuing Instead of Strict Per-Tenant Queues? . . . 7
- 4.2 Protobuf Configuration (`FairQueuePolicyConfig`) . . . 7
- 4.3 Internal Data Structures of `FairQueue<ItemType>` . . . 8
- 4.4 Core Operations and Complexity Analysis . . . 9
  - 4.4.1 Zero-Allocation Round-Robin Turn Advancement via `std::list::splice` . . . 9
- 4.5 Class Hashing, Type Seeding, Default Bucket Isolation, and Seed Perturbation . . . 10

5 Comparison with Existing Envoy QoS Mechanisms & Extensibility **11**
5.1 How Upstream Queue Policies Fit into Envoy's Traffic Management Stack 11
5.2 Extensibility Hooks in `QueueBase` for Future Queue Policies 11

6 Benchmark Methodology, Queueing-Theory Analysis, and Results **13**
6.1 Experimental Architecture and Workload Configuration 13
6.2 Queueing-Theory and Little's Law Derivation of the Benchmark Numbers 13
6.2.1 1. Baseline Phase ($t \in [0, 10)$ s and $t \in (25, 30]$ s) 13
6.2.2 2. Burst Phase Under FIFO (`no_queue` and `fifo`, $t \in [10, 25]$ s) 13
6.2.3 3. Burst Phase Under Stochastic Fair Queuing (`fair_queue`, $t \in [10, 25]$ s) . . . 15
6.3 Empirical Results Summary Table 15
6.4 Visual Analysis of Benchmark Graphs 16

7 Production Configuration Reference **18**
7.1 Key Source Files Reference 19

1 Problem Statement: Head-of-Line Blocking in Upstream Connection Pools

Envoy routes downstream requests to upstream clusters through protocol-specific connection pools (`Http::Http1::ProdConnPoolImpl`, `Http::Http2::ProdConnPoolImpl`, `Http::Http3::ProdConnPoolImpl`, and `Tcp::ConnPoolImpl`). All connection pools inherit their stream-queuing and capacity-accounting state machine from `ConnectionPool::ConnPoolImplBase` (`source/common/conn_pool/conn_pool_base.h`).

1.1 When Requests Enter the Pending Stream Queue

When `ConnPoolImplBase::newStreamImpl()` receives a stream request, it attempts to attach the stream immediately to an existing connection in `ready_clients_` (or `early_data_clients_` for 0-RTT). A request cannot be dispatched immediately when all existing connections are in the `Busy` or `Connecting` state:

- **HTTP/1.1**: Each connection handles at most one active request at a time (no pipelining). Once active connections reach `circuit_breakers.thresholds.max_connections`, no new connections can be opened.
- **HTTP/2 and HTTP/3**: Each connection multiplexes requests up to `max_concurrent_streams`. When active streams saturate existing connections and either new connections are still completing TCP/TLS/QUIC handshakes or `max_connections` is reached, new requests cannot be attached immediately.

When no ready client exists, `ConnPoolImplBase::newStreamImpl()` checks the cluster's `max_pending_requests` circuit breaker. If capacity remains, the pool wraps the request in a `PendingStream` object and inserts it into the pending stream queue until `onUpstreamReady()` fires (triggered when an in-flight stream closes in `onStreamClosed()` or a connecting client completes its handshake in `onConnectionEvent()`).

1.2 The Multi-Tenant Noisy-Neighbor Failure Mode

Before the `envoy.queue_policy` extension point, `ConnPoolImplBase` stored pending requests in a single flat `std::list<PendingStreamPtr>` serviced in strict arrival (FIFO) order. In a multi-tenant environ-

ment where multiple downstream clients share an upstream cluster, FIFO creates three compounding failure modes during a single-tenant traffic spike:

1. **Latency Coupling (HoL Blocking)**: If a noisy tenant sends requests at rate λ_{noisy} exceeding upstream service rate μ, the shared FIFO queue builds a standing backlog of depth Q. Every request from a quiet, well-behaved tenant ($\lambda_{\text{wb}} \ll \mu$) arriving at the pool must wait behind all Q queued requests, experiencing the exact same queueing delay $W_q = Q/\mu$ as the noisy tenant.

2. **Self-Amplifying Queue Accumulation**: By Little's Law ($L = \lambda \cdot W$), forcing well-behaved requests to wait for duration W causes each well-behaved tenant to accumulate $\lambda_{\text{wb}} \cdot W$ of its own requests inside Envoy's pending queue. This inflates `upstream_rq_pending_active` and accelerates `max_pending_requests` circuit-breaker overflow (`503 UC`) for all tenants.

3. **Upstream Throughput Starvation**: In a FIFO queue, each tenant receives upstream service bandwidth proportional to its arrival rate ($\lambda_i / \sum \lambda_k$). A tenant sending 500 RPS alongside three tenants sending 60 RPS captures $500/680 \approx 73.5\%$ of upstream connection slots if unconstrained by downstream connection limits, starving steady tenants.

2 Architecture of the `envoy.queue_policy` Extension Point

The `envoy.queue_policy` framework decouples **stream lifetime ownership** in `ConnPoolImplBase` from **pending stream dispatch ordering** via a pluggable strategy interface.

2.1 Cluster Proto Configuration and Load-Time Resolution

Clusters configure their pending request queue policy via field 62 of `envoy.config.cluster.v3.Cluster` (`api/envoy/config/cluster/v3/cluster.proto`):

```
message Cluster {
  // Queueing policies for the cluster.
  message QueuingPolicies {
    // Queue policy for cluster pending requests. If unset, Envoy uses the FIFO
        queue policy.
    // [#extension-category: envoy.queue_policy]
    core.v3.TypedExtensionConfig pending_rq_policy = 1;
  }

  // Queueing policies for the cluster (e.g. the queue policy for cluster pending
      requests).
  QueuingPolicies queuing_policies = 62;
}
```

Listing 1: Cluster proto definition for pending request queuing policies.

Envoy creates connection pools dynamically on each worker thread for every upstream host, routing priority, protocol, transport socket option, and downstream filter-state hash. Resolving the factory registry and unpacking `google.protobuf.Any` on every connection pool instantiation would add overhead to the data plane and delay configuration errors until traffic arrives.

Instead, `ClusterInfoImpl` resolves and validates the policy once at cluster load time via `resolvePendingRqQueueP` (`source/common/upstream/upstream_impl.cc`):

1. Looks up `QueuePolicyFactory<ConnectionPool::PendingStream>` in the static `Registry::FactoryRegistry`

2. Translates the opaque `TypedExtensionConfig` into the extension's typed protobuf message using `Config::Utility::translateOpaqueConfig()`.

3. Instantiates a test queue policy via `factory->createQueuePolicy()` during cluster initialization so invalid configurations are rejected immediately (NACKed via xDS).

4. Stores the resolved factory pointer, validated protobuf message, and cluster stat prefix in `ClusterInfo::PendingS` on the immutable `ClusterInfo`.

When a worker thread constructs a `ConnPoolImplBase`, `createPendingStreamQueue()` (`source/common/conn_po` invokes `factory.createQueuePolicy(*queue_policy->config_, ...)` directly without registry lookups or Any unpacking. If `queuing_policies` is unset, `createPendingStreamQueue()` defaults to `FifoQueue<PendingStr`

2.2 Core Interfaces: `QueueBase` and `QueueItemMetadata`

All queue policies implement `Extensions::QueuePolicy::QueueBase<ItemType>` defined in `source/common/queue_p`

```cpp
// Metadata associated with an item when it is added to a queue.
struct QueueItemMetadata {
  MonotonicTime enqueue_time_;
  OptRef<const StreamInfo::StreamInfo> stream_info_;
};

template <class ItemType> class QueueBase {
public:
  virtual ~QueueBase() = default;

  virtual size_t size() const PURE;
  virtual bool empty() const PURE;

  // Adds an item to the queue. The caller retains ownership and must keep the item alive
  // until it is removed with pop() or remove().
  virtual void add(ItemType& item, QueueItemMetadata metadata) PURE;

  // Returns the next item to be dequeued. Illegal to call on an empty queue.
  virtual const ItemType& peek() const PURE;
  virtual ItemType& peek() PURE;

  // Removes the next item from the queue without destroying it.
  virtual void pop() PURE;

  // Removes a specific item from the queue without destroying it.
  virtual void remove(ItemType& item) PURE;

  virtual bool isOverloaded() const PURE;

  // Iterates over queued items in dequeue order. The callback may safely call remove().
  virtual void forEach(absl::FunctionRef<bool(ItemType&)> cb) PURE;
};
```

Listing 2: `QueueItemMetadata` and `QueueBase<ItemType>` interface (`queue_policy_base.h`).

2.3 Dual-Container Ownership Model in `ConnPoolImplBase`

A central design decision in `ConnPoolImplBase` is the separation of **memory ownership** from **ordering policy**:

- **Ownership Container (`std::list<PendingStreamPtr> pending_streams_`)**: Owns the heap-allocated `PendingStream` instances via `std::unique_ptr<PendingStream>`. Because `PendingStream`

inherits from `LinkedObject<PendingStream>`, each `PendingStream` stores its own `std::list` iterator, allowing `stream.removeFromList(pending_streams_)` to unlink and return ownership in $O(1)$ time regardless of where the stream sits in the list.

- **Ordering Policy (`PendingStreamQueuePtr pending_stream_queue_`)**: Holds non-owning references (`PendingStream&`) and dictates which stream is dispatched next via `peek()` and `pop()`.

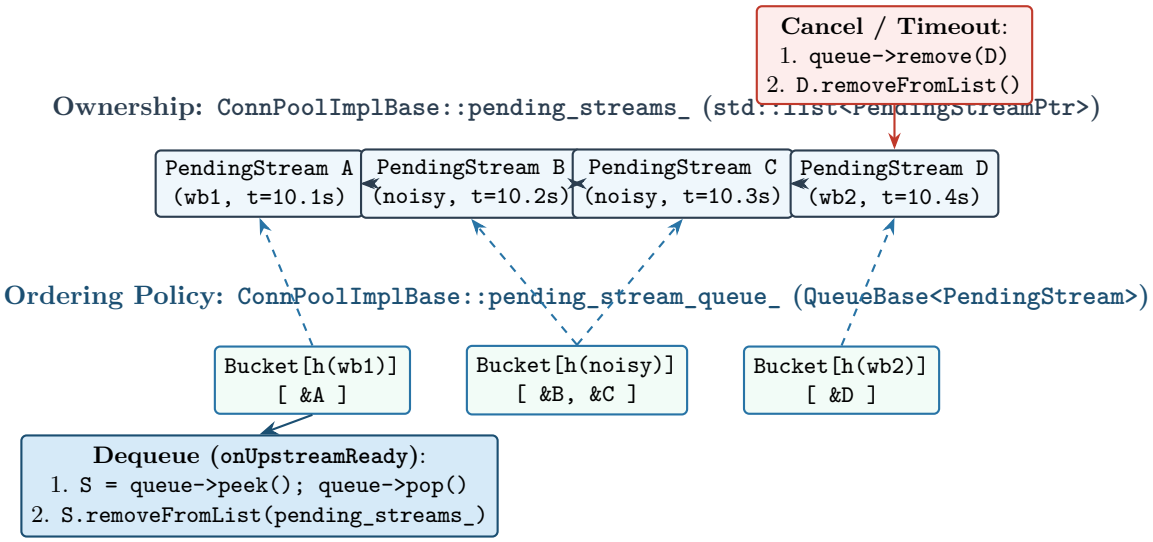


Figure 1: Dual-container architecture in `ConnPoolImplBase`: `pending_streams_` owns `PendingStream` memory via intrusive `LinkedObject` pointers, while `pending_stream_queue_` maintains non-owning references to control dispatch order.

Figure 1 illustrates how the four core connection pool lifecycle operations interact with both containers while preserving the strict invariant `ASSERT(pending_stream_queue_->size() == pending_streams_.size())`:

1. **Stream Enqueue (`addPendingStream`)**: Moves the `PendingStreamPtr` into the back of `pending_streams_`, calls `pending_stream_queue_->add(stream, {dispatcher_.timeSource().monotonicTime(), stream.context().stream_info_})`, increments `cluster_connectivity_state_.incrPendingStreams(1)`, and updates the `upstream_queue_overloaded` gauge.

2. **Upstream Ready Dispatch (`onUpstreamReady` → `popPendingStream`)**: While `hasPendingStreams() && !ready_clients_.empty()`, `popPendingStream()` calls `pending_stream_queue_->peek()` and `pop()` to select the next `PendingStream&` according to the active policy, then calls `stream.removeFromList(pend` to transfer ownership out of `pending_streams_` before attaching the stream to `*ready_clients_.front()`.

3. **Mid-Queue Cancellation (`onPendingStreamCancel` → `removePendingStream`)**: If a downstream client resets the HTTP request or a router timeout fires while the stream is still queued, `PendingStream::cancel()` invokes `removePendingStream(stream)`. This calls `pending_stream_queue_->re` followed by `stream.removeFromList(pending_streams_)`. Because cancellations can happen at high rates during downstream timeouts, `QueueBase::remove(ItemType&)` must execute in $O(1)$ time without scanning the queue.

4. **Connect Failure Purge (`purgePendingStreams`)**: If an upstream connection attempt fails before handshake completion, `purgePendingStreams()` drains `pending_stream_queue_` via `popPendingStream()` into a temporary list `pending_streams_to_purge_` before invoking `onPoolFailure()` callbacks, ensuring that any synchronous retry streams enqueued by the failure callback are not purged in the same pass.

3 End-to-End Request Classification Pipeline

Because `ConnPoolImplBase` resides in `source/common/conn_pool/` and is shared across HTTP/1.1, HTTP/2, HTTP/3, and TCP connection pools, `QueueBase` cannot depend directly on `Http::RequestHeaderMap`. Instead, `QueueItemMetadata` receives an `OptRef<const StreamInfo::StreamInfo>`, allowing any downstream filter to classify traffic into `StreamInfo::dynamicMetadata()` before the request reaches the router.

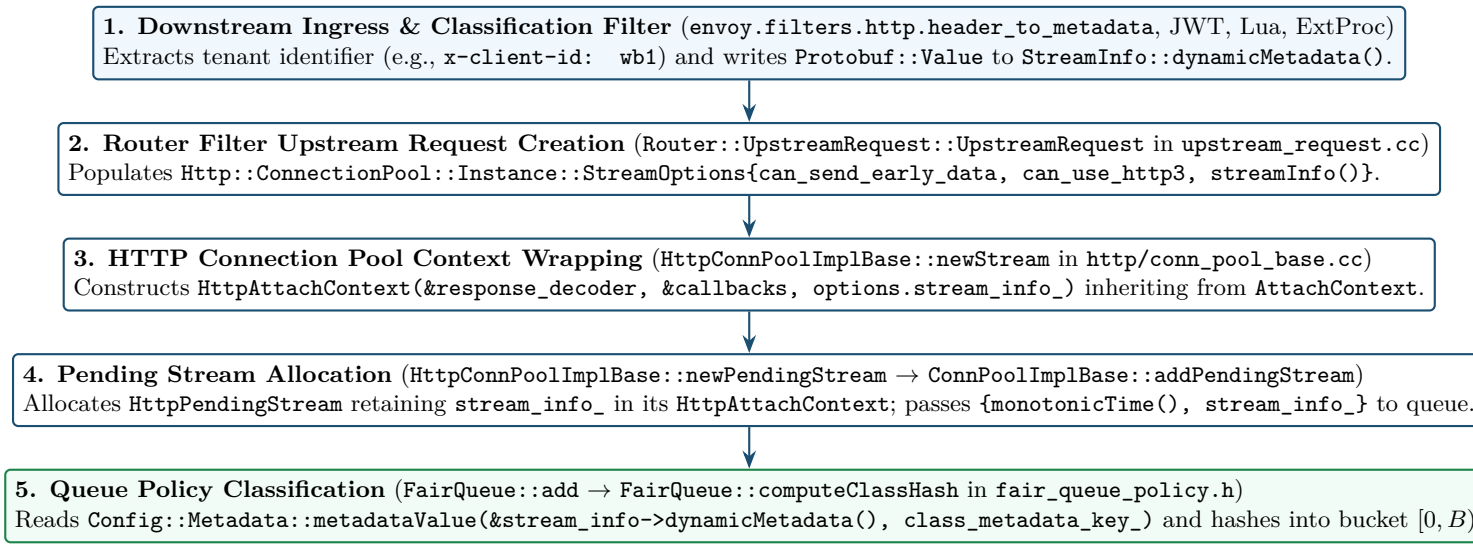


Figure 2: End-to-end plumbing of `StreamInfo` from downstream HTTP classification filters to `FairQueue::add()`.

Figure 2 traces the exact five-step path across Envoy's codebase:

1. **Metadata Population**: An HTTP filter such as `envoy.filters.http.header_to_metadata` inspects incoming request headers (e.g., `x-client-id` or `x-tenant-tier`) and writes a `Protobuf::Value` into `StreamInfo::dynamicMetadata()` under a filter namespace and key. Any other filter (JWT authentication claims, RBAC, ExtAuthz, Lua, or Wasm) can populate this metadata key.

2. **Router `StreamOptions` Initialization**: In `source/common/router/upstream_request.cc`, `UpstreamRequest` initializes `stream_options_` with an optional reference to the downstream `StreamInfo`:

```
stream_options_({can_send_early_data, can_use_http3,
                 makeOptRef<const
                     StreamInfo::StreamInfo>(parent_.callbacks()->streamInfo())})
```

3. **`AttachContext` Propagation**: `ConnectionPool::AttachContext` (`source/common/conn_pool/conn_pool_b` carries `OptRef<const StreamInfo::StreamInfo> stream_info_`. `HttpConnPoolImplBase::newStream()` forwards `options.stream_info_` into `HttpAttachContext`, and `newPendingStream()` copies that `OptRef` into `HttpPendingStream::context_`.

4. **`QueueItemMetadata` Construction**: When `ConnPoolImplBase::addPendingStream()` enqueues the `PendingStream`, it constructs `QueueItemMetadata{dispatcher_.timeSource().monotonicTime(), stream.context().stream_info_}` and passes it to `pending_stream_queue_->add()`.

4 The Fair Queue Policy (`envoy.queue_policy.fair_queue`) & SFQ Data Structure

The `envoy.queue_policy.fair_queue` extension (`source/extensions/queue_policy/fair_queue/fair_queue_po` implements **Stochastic Fair Queuing (SFQ)**, originally introduced by Paul E. McKenney (1990) for packet scheduling and adapted here for connection pool request scheduling with $O(1)$ mid-queue cancellation.

4.1 Why Stochastic Fair Queuing Instead of Strict Per-Tenant Queues?

A naive fair-queuing implementation might maintain a dynamic hash map from tenant ID string to a dedicated per-tenant FIFO queue (`absl::flat_hash_map<std::string, std::list<ItemType*>>`). SFQ was chosen over strict per-tenant queues for three operational reasons:

1. **Bounded Memory Footprint Under Unbounded Tenant Cardinality**: In a strict per-tenant map, an attacker or misconfigured client sending requests with millions of unique tenant IDs forces Envoy to allocate and track millions of map entries or run background garbage-collection sweeps. SFQ pre-allocates a fixed vector of B buckets (`num_buckets`, default `1024`) at connection pool construction time. Memory scales strictly with the number of currently queued items ($\leq$ `max_pending_requests`) plus $O(B)$ fixed table overhead.

2. **Zero Per-Tenant State Management**: SFQ never stores tenant ID strings in the queue. It hashes the `Protobuf::Value` on arrival into a bucket index in $[0, B)$ and discards the key.

3. **Bounded Collision Penalty via Seed Perturbation**: With $B = 1024$ buckets and K concurrent active tenants ($K \ll B$), the probability that a specific well-behaved tenant collides with a single noisy neighbor in any given interval is $1/B \approx 0.098\%$. Even if a collision occurs, periodic hash seed perturbation (`perturbation_interval`, default `1s`) rotates the hash mapping so that no two tenants remain persistently coupled in the same bucket.

4.2 Protobuf Configuration (`FairQueuePolicyConfig`)

The configuration proto (`api/envoy/extensions/queue_policy/fair_queue/v3/fair_queue.proto`) exposes three fields:

```
message FairQueuePolicyConfig {
  // Specifies the dynamic metadata key in StreamInfo::dynamicMetadata() used to
      classify
  // queued requests into fair-queue buckets.
  type.metadata.v3.MetadataKey class_metadata_key = 1
      [(validate.rules).message = {required: true}];

  // The number of hash buckets used to partition queued requests across classes.
  // Defaults to 1024 if unset.
  google.protobuf.UInt32Value num_buckets = 2 [(validate.rules).uint32 = {gt: 0}];

  // The time interval at which the bucket hash function perturbs its seed to break
      hash
  // collisions among classes. Defaults to 1 second if unset. Setting to 0s
      disables perturbation.
  google.protobuf.Duration perturbation_interval = 3 [(validate.rules).duration =
      {gte {}}];
}
```

Listing 3: `FairQueuePolicyConfig` protobuf definition.

4.3 Internal Data Structures of `FairQueue<ItemType>`

To support $O(1)$ `add()`, $O(1)$ `peek()`, $O(1)$ `pop()`, and $O(1)$ arbitrary mid-queue `remove()`, `FairQueue<ItemType>` coordinates four internal containers:

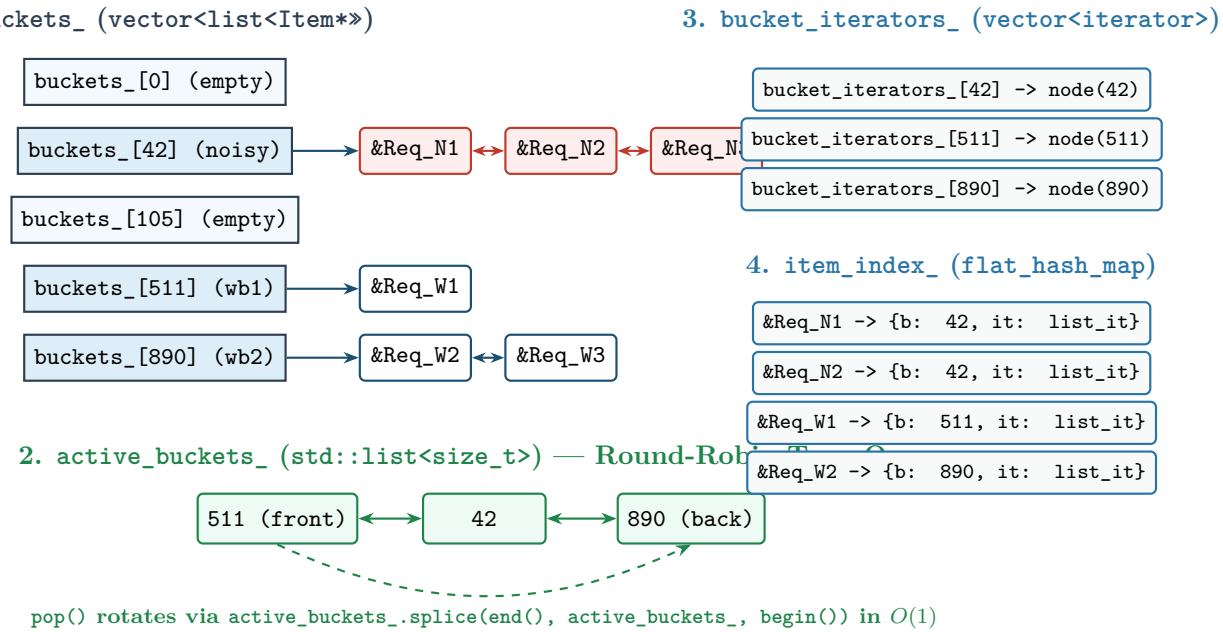


Figure 3: Internal data structures of `FairQueue<ItemType>`: `buckets_` stores per-bucket FIFO queues, `active_buckets_` tracks non-empty buckets in round-robin order, `bucket_iterators_` enables $O(1)$ bucket deactivation, and `item_index_` enables $O(1)$ arbitrary item removal.

As shown in Figure 3, the four member containers in `FairQueue<ItemType>` work together as follows:

1. `std::vector<ItemList> buckets_` (where `ItemList = std::list<ItemType*>`): A pre-allocated vector of size `num_buckets_`. Each element is a doubly-linked FIFO list of item pointers belonging to that hash bucket.

2. `BucketList active_buckets_` (where `BucketList = std::list<size_t>`): A doubly-linked list containing *only* the bucket indices $[0, \texttt{num_buckets_})$ that are currently non-empty. Keeping only non-empty buckets in `active_buckets_` guarantees that `peek()` and `pop()` never scan empty buckets; dequeue is $O(1)$ regardless of how large `num_buckets_` is configured.

3. `std::vector<typename BucketList::iterator> bucket_iterators_`: Pre-allocated vector of size `num_buckets_`. Whenever `buckets_[b]` transitions from empty to non-empty in `add()`, its index `b` is appended to `active_buckets_`, and the resulting `BucketList::iterator` is saved in `bucket_iterators_[b]`. When `buckets_[b]` becomes empty (either via `pop()` or mid-queue `remove()`), `active_buckets_.erase(bucket_iterators_[b])` deactivates the bucket in $O(1)$ time.

4. `absl::flat_hash_map<ItemType*, ItemLocation> item_index_`: Maps each queued `ItemType*` pointer to an `ItemLocation{size_t bucket_idx_, typename ItemList::iterator item_it_}`. This provides $O(1)$ total queue size (`item_index_.size()`) and $O(1)$ direct erasure from `buckets_[bucket_idx_` when `remove(ItemType&)` is called on an arbitrary mid-queue item.

Method	Complexity	Internal Execution Mechanics
`size()` / `empty()`	$O(1)$	Queries `item_index_.size()` and `item_index_.empty()`.
`add(item, meta)`	$O(1)$ avg	Perturbs hash seed if interval elapsed; hashes metadata via `xxHash64` to `bucket_idx`; appends `bucket_idx` to `active_buckets_` if bucket was empty; inserts `&item` at back of `buckets_[bucket_idx]`; inserts `ItemLocation` into `item_index_`.
`peek()`	$O(1)$	Reads `bucket_idx = active_buckets_.front()` and returns `*buckets_[bucket_idx].front()`.
`pop()`	$O(1)$	Calls `remove(*buckets_[bucket_idx].front())`. If `buckets_[bucket_idx]` remains non-empty, splices `active_buckets_.begin()` to `active_buckets_.end()` in $O(1)$ with zero heap allocations and without invalidating `bucket_iterators_[bucket_idx]`.
`remove(item)`	$O(1)$ avg	Looks up `&item` in `item_index_`; erases `item_it_` from `buckets_[bucket_idx]`; erases entry from `item_index_`; if `buckets_[bucket_idx].empty()`, erases `bucket_iterators_[bucket_idx]` from `active_buckets_`.
`forEach(cb)`	$O(N)$	Materializes a round-robin dequeue-order snapshot via per-bucket cursors, then invokes `cb(*item)` for each item still present in `item_index_`.

Table 1: Time complexity and mechanics of `FairQueue<ItemType>` operations (N = queued items).

4.4 Core Operations and Complexity Analysis

4.4.1 Zero-Allocation Round-Robin Turn Advancement via `std::list::splice`

Examine `FairQueue::pop()` and `FairQueue::remove()` in `source/extensions/queue_policy/fair_queue/fair_q`

```
void pop() override {
  ASSERT(!active_buckets_.empty());
  const size_t bucket_idx = active_buckets_.front();
  ASSERT(!buckets_[bucket_idx].empty());
  remove(*buckets_[bucket_idx].front());
  if (!buckets_[bucket_idx].empty()) {
    active_buckets_.splice(active_buckets_.end(), active_buckets_,
        active_buckets_.begin());
  }
}

void remove(ItemType& item) override {
  auto entry = item_index_.find(&item);
  ASSERT(entry != item_index_.end());
  const size_t bucket_idx = entry->second.bucket_idx_;
  ItemList& bucket = buckets_[bucket_idx];
  bucket.erase(entry->second.item_it_);
  item_index_.erase(entry);
  if (bucket.empty()) {
    active_buckets_.erase(bucket_iterators_[bucket_idx]);
  }
}
```

Listing 4: Round-robin dequeue and arbitrary item removal in `FairQueue<ItemType>`.

Two implementation details here are particularly important:

1. **Iterator Stability via `splice()`**: When `pop()` removes the front item of `buckets_[bucket_idx]` and the bucket still has remaining items, `pop()` rotates `bucket_idx` from the front of `active_buckets_` to the back using `active_buckets_.splice(active_buckets_.end(), active_buckets_, active_buckets_`. Unlike `pop_front()` followed by `push_back()`, `std::list::splice` relinks the existing list node in-place without allocating memory and **without invalidating the iterator stored in `bucket_iterators_[bucke`**.

2. **Turn Preservation on Mid-Queue Cancellation**: If the head item of the front bucket is cancelled via `remove(item)` (rather than dequeued via `pop()`) while another item from that same tenant is still waiting behind it in the bucket, `remove()` erases only the cancelled item and does *not* rotate `active_buckets_`. Thus, a tenant whose front request times out does not lose its current round-robin turn for its next waiting request.

4.5 Class Hashing, Type Seeding, Default Bucket Isolation, and Seed Perturbation

`FairQueue::computeClassHash()` computes a 64-bit hash from `QueueItemMetadata` using `HashUtil::xxHash64`:

```
void maybePerturbSeed(MonotonicTime enqueue_time) {
  if (perturbation_interval_.count() <= 0) {
    return;
  }
  if (!last_perturbation_time_.has_value()) {
    last_perturbation_time_ = enqueue_time;
    return;
  }
  if (enqueue_time >= *last_perturbation_time_ + perturbation_interval_) {
    last_perturbation_time_ = enqueue_time;
    ++perturbation_seed_;
  }
}

uint64_t computeClassHash(const QueueItemMetadata& metadata) const {
  if (!metadata.stream_info_.has_value()) {
    return HashUtil::xxHash64Value(uint32_t{0}, perturbation_seed_);
  }
  const Protobuf::Value& value = Envoy::Config::Metadata::metadataValue(
      &metadata.stream_info_->dynamicMetadata(), class_metadata_key_);
  switch (value.kind_case()) {
  case Protobuf::Value::kStringValue: {
    const uint64_t type_seed =
        HashUtil::xxHash64Value(static_cast<uint32_t>(value.kind_case()),
            perturbation_seed_);
    return HashUtil::xxHash64(value.string_value(), type_seed);
  }
  case Protobuf::Value::kNumberValue: {
    const uint64_t type_seed =
        HashUtil::xxHash64Value(static_cast<uint32_t>(value.kind_case()),
            perturbation_seed_);
    return HashUtil::xxHash64Value(value.number_value(), type_seed);
  }
  case Protobuf::Value::kBoolValue: {
    const uint64_t type_seed =
        HashUtil::xxHash64Value(static_cast<uint32_t>(value.kind_case()),
            perturbation_seed_);
    return HashUtil::xxHash64Value(value.bool_value(), type_seed);
  }
  case Protobuf::Value::kStructValue:
  case Protobuf::Value::kListValue: {
    const uint64_t type_seed =
```

```
40          HashUtil::xxHash64Value(static_cast<uint32_t>(value.kind_case()),
                perturbation_seed_);
41      return HashUtil::xxHash64Value(static_cast<uint64_t>(ValueUtil::hash(value)),
            type_seed);
42    }
43    case Protobuf::Value::kNullValue:
44    case Protobuf::Value::KIND_NOT_SET:
45      return HashUtil::xxHash64Value(uint32_t{0}, perturbation_seed_);
46    }
47    return HashUtil::xxHash64Value(uint32_t{0}, perturbation_seed_);
48  }
```

Listing 5: Two-stage `xxHash64` class hashing and unclassified traffic isolation.

Three subtle design choices in `computeClassHash()` and `maybePerturbSeed()` prevent production edge cases:

1. **All `Protobuf::Value` Kinds with Type-Domain Separation**: `computeClassHash()` supports strings, numbers, booleans, nested structs, and lists. It first hashes `value.kind_case()` with `perturbation_seed_` to derive `type_seed`, and then hashes the value payload using `type_seed`. This guarantees that different types with similar byte representations (e.g., number `0`, boolean `false`, and missing metadata) hash into distinct domains.

2. **Single Default Bucket for Unclassified Traffic**: If a request lacks `StreamInfo`, does not carry the configured `class_metadata_key`, or has a `NullValue`, `computeClassHash()` maps it to `HashUtil::xxHash64Value(uint32_t{0}, perturbation_seed_)`. All unclassified requests share **one single default bucket**. If unclassified requests were hashed by request ID or allowed to bypass bucketing, an unauthenticated flood omitting the tenant header would scatter across all 1024 buckets and starve authenticated tenants. Mapping all untagged traffic to one default class confines untagged floods to $1/N_{\text{active}}$ of upstream capacity.

3. **Lazy Timestamp-Driven Perturbation (Zero Timer Overhead)**: Because a busy Envoy instance can maintain thousands of connection pools across worker threads, creating an `Event::Timer` per connection pool to rotate seeds would add unnecessary event-loop timer overhead. Instead, `maybePerturbSeed()` checks `enqueue_time >= *last_perturbation_time_ + perturbation_interval_` lazily during `add()`. When the interval elapses, `++perturbation_seed_` rotates the hash function for newly arriving requests while leaving already-queued requests in their current buckets so per-tenant FIFO ordering is preserved for in-flight items.

5 Comparison with Existing Envoy QoS Mechanisms & Extensibility

5.1 How Upstream Queue Policies Fit into Envoy's Traffic Management Stack

5.2 Extensibility Hooks in `QueueBase` for Future Queue Policies

The `QueueBase<ItemType>` interface was designed to support a broader family of active queue management (AQM) policies beyond FIFO and SFQ:

- **Sojourn-Time Tracking (`QueueItemMetadata::enqueue_time_`)**: Every `add()` call receives `dispatcher_.timeSource().monotonicTime()`. Future policies such as **CoDel (Controlled Delay)**, **PIE**, or **LIFO-under-overload** can compute exact queue sojourn time ($t_{\text{now}} - $`enqueue_time_`) upon dequeue or inspection.

- **Cluster Overload Gauge (`QueueBase::isOverloaded()` → `upstream_queue_overloaded`)**: On every `addPendingStream()` and `onUpstreamReady()`, `ConnPoolImplBase::updateQueueOverloadedGauge`

Mechanism	Where It Operates	Primary Function	Limitation Without `FairQueue`
Circuit Breakers (`max_pending_requests`, `max_connections`)	Upstream Cluster / Priority	Hard cap on aggregate active connections and pending queue depth; rejects excess with `503 UC`.	Tenant-blind: a noisy neighbor fills `max_pending_requests` and causes well-behaved tenants to be rejected or delayed.
Routing Priority (`HIGH` vs. `DEFAULT`)	Route → Separate `ConnPoolImplBase`	Allocates completely separate connection pools and circuit breaker budgets per priority class.	Only 2 static tiers; splits connection pool reuse across priorities; cannot isolate hundreds or thousands of dynamic tenants.
Local / Global Rate Limit Filter	Downstream HTTP Filter Chain	Enforces static token-bucket rate limits per descriptor before routing.	**Not work-conserving**: throttles bursts even when the upstream cluster has spare capacity; requires tuning per-tenant rate limits.
Load Shed Points (`OverloadManager`)	Server-wide Resource Monitors	Sheds connections/streams when host CPU or memory crosses critical thresholds.	Protects Envoy process memory/CPU, not upstream connection pool fairness across tenants.
Fair Queue Policy (`envoy.queue_policy.fair_queue`)	Inside `ConnPoolImplBase` (`pending_stream_queue_`)	Isolates dynamic tenant classes into B buckets and schedules upstream connection slots in round-robin order.	**Work-conserving & zero rate-tuning**: tenants can burst using 100% of idle upstream capacity, and are isolated to $1/N_{\text{active}}$ share only during contention.

Table 2: Comparison of `envoy.queue_policy.fair_queue` with existing Envoy traffic management controls.

checks `pending_stream_queue_->isOverloaded()` and transitions the cluster's `upstream_queue_overloaded` gauge. Policies that detect standing queues (e.g., CoDel when minimum sojourn time exceeds target delay over an interval) can report overload directly to cluster metrics and load-shedding controllers.

- **Dequeue-Order Inspection & Eviction (`QueueBase::forEach()`)**: Because `forEach()` visits queued items in dequeue order and explicitly permits the callback to invoke `remove(item)`, connection pools or composite policies can scan and evict expired or lower-priority items cleanly.

6 Benchmark Methodology, Queueing-Theory Analysis, and Results

To quantify the impact of `envoy.queue_policy.fair_queue` under controlled, repeatable upstream contention, the benchmark suite in `tools/queue_policy_test/` drives multi-client Nighthawk HTTP traffic through `envoy-static` against a capacity-constrained upstream backend.

6.1 Experimental Architecture and Workload Configuration

Figure 4 and Table 3 summarize the benchmark topology configured in `tools/queue_policy_test/scenario.yaml`:

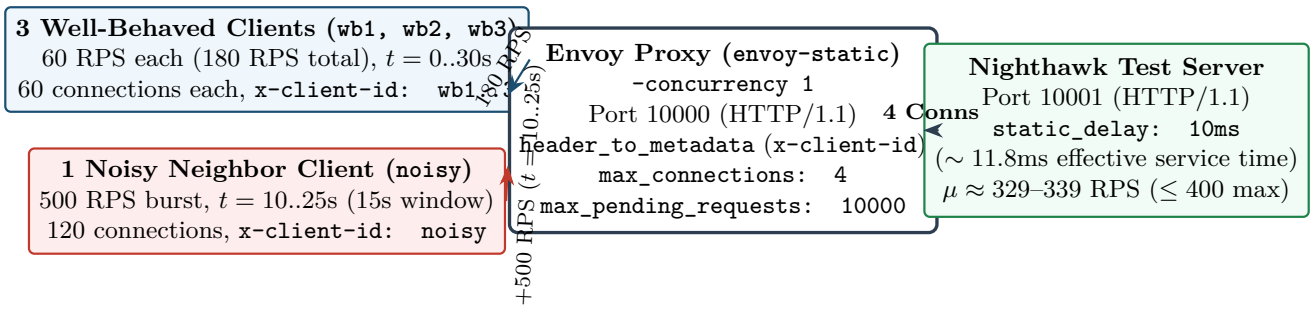


Figure 4: Benchmark topology in `tools/queue_policy_test/`: four Nighthawk client processes generate steady and burst traffic through a single-worker Envoy instance to a 4-connection HTTP/1.1 backend.

6.2 Queueing-Theory and Little's Law Derivation of the Benchmark Numbers

The empirical latency and queue-depth measurements in this benchmark follow directly from Little's Law ($L = \lambda \cdot W$) and connection-bounded client concurrency:

6.2.1 1. Baseline Phase ($t \in [0, 10)$ s and $t \in (25, 30]$ s)

Only `wb1`, `wb2`, and `wb3` are active, generating an aggregate arrival rate of $\lambda_{\text{total}} = 3 \times 60 = 180$ RPS. Because $\lambda_{\text{total}} = 180$ RPS $< \mu \approx 335$ RPS (utilization $\rho \approx 0.54$), the 4 upstream HTTP/1.1 connections absorb incoming requests with negligible queuing. Client-observed latency remains near the single-request service time ($W \approx 11.8$ ms).

6.2.2 2. Burst Phase Under FIFO (`no_queue` and `fifo`, $t \in [10, 25]$ s)

At $t = 10$ s, `noisy` begins attempting 500 RPS, pushing offered load to $\lambda_{\text{offered}} = 180 + 500 = 680$ RPS $> \mu \approx 329$ RPS.

Component	Parameter	Value & Rationale
Envoy Proxy	`-concurrency`	`1` worker thread (isolates a single `ConnPoolImplBase` instance).
	`max_connections`	`4` upstream HTTP/1.1 connections (1 active request per connection).
	`max_pending_requests`	`10000` (prevents circuit-breaker shedding so pure queuing delay is measured).
	Classification Filter	`envoy.filters.http.header_to_metadata` mapping `x-client-id` → `client_id`.
Upstream Server	`static_delay`	`0.010s` (10 ms) in `nighthawk_test_server`. With $\sim$ 1.8 ms socket/HTTP framing overhead, effective service time is $S \approx 11.8$ ms ($\mu \approx 329$–339 RPS).
Clients `wb1..wb3`	Rate & Concurrency	60 RPS each (180 RPS combined), 60 HTTP/1.1 connections each, active $t = 0..30$ s.
Client `noisy`	Rate & Concurrency	500 RPS target rate, 120 HTTP/1.1 connections, active during burst $t = 10..25$ s.
Policies Tested	`no_queue`	Baseline default (`Cluster.queuing_policies` omitted; uses internal `FifoQueue`).
	`fifo`	Explicit `envoy.queue_policy.fifo` extension.
	`fair_queue`	`envoy.queue_policy.fair_queue` (`num_buckets: 1024`, `perturbation_interval: 1s`).

Table 3: Benchmark configuration parameters defined in `tools/queue_policy_test/scenario.yaml`.

- **Noisy Neighbor Connection Saturation**: Because the upstream pool cannot drain 680 RPS, the queue builds rapidly until all $C_{\text{noisy}} = 120$ HTTP/1.1 connections of the `noisy` client have an in-flight request waiting inside Envoy. Once its 120 connections are occupied, `noisy` becomes a closed-loop client whose effective arrival rate is governed by round-trip latency W: $\lambda_{\text{noisy, eff}} = C_{\text{noisy}}/W$.
- **Shared FIFO Latency Equilibrium**: Because FIFO places all requests into a single line, all four clients experience the same steady-state round-trip time W_{FIFO} during the burst. The three well-behaved clients remain open-loop at $\lambda_{\text{wb}} = 60$ RPS each (since $60 \cdot W_{\text{FIFO}} < 60$ connections).
- **Solving for W_{FIFO} and Queue Depth Q_{FIFO}**: Total completed throughput during the burst is bounded by the upstream service rate $\mu \approx 329$ RPS. Since well-behaved clients inject ~ 171 RPS (3×57.1 RPS accounting for boundary ramp) and `noisy` receives the remaining capacity:

  $$\lambda_{\text{noisy, eff}} = \mu - 171 \approx 329 - 171 = 158 \text{ RPS}$$

  In equilibrium, total in-flight requests inside Envoy ($L_{\text{total}} = Q_{\text{FIFO}} + 4$ active) across all clients satisfy Little's Law:

  $$W_{\text{FIFO}} = \frac{L_{\text{total}}}{\mu} \approx 667 \text{ ms} \quad \Longrightarrow \quad L_{\text{total}} = 329 \text{ req/s} \times 0.667 \text{ s} \approx 219 \text{ requests} \quad (Q_{\text{FIFO}} \approx 204\text{–}218 \text{ pending})$$

- **Why Well-Behaved Requests Make Up Over Half of the FIFO Queue**: By Little's Law, at $W_{\text{FIFO}} = 0.667$ s, each well-behaved client has $L_{\text{wb}} = 60 \text{ RPS} \times 0.667 \text{ s} = 40$ requests trapped in Envoy's queue ($3 \times 40 = \mathbf{120}$ well-behaved requests, or **58% of the entire pending queue**), alongside $\sim$ 88–98 requests from `noisy`! Furthermore, during the onset of the burst ($t = 10..11$ s), absorbing 40 requests per well-behaved client into the standing backlog causes their completed throughput to dip from 60 RPS to an average of **57.1** RPS.

6.2.3 3. Burst Phase Under Stochastic Fair Queuing (`fair_queue`, $t \in [10, 25]$ s)

Under `envoy.queue_policy.fair_queue`, `wb1`, `wb2`, `wb3`, and `noisy` hash into four distinct buckets ($N_{\text{active}} = 4$):

- **Guaranteed Per-Bucket Service Rate**: Because `FairQueue::pop()` visits non-empty buckets in round-robin order, each of the 4 active buckets is guaranteed at least:

$$\mu_{\text{fair}} = \frac{\mu}{N_{\text{active}}} \approx \frac{329 \text{ RPS}}{4} = 82.3 \text{ RPS}$$

- **Zero Standing Queue for Well-Behaved Clients**: Each well-behaved client sends $\lambda_{\text{wb}} = 60$ RPS $< \mu_{\text{fair}} = 82.3$ RPS (bucket utilization $\rho_{\text{wb}} = 60/82.3 = 0.73 < 1$). Because their arrival rate is strictly below their guaranteed service share, the buckets for `wb1`, `wb2`, and `wb3` **never accumulate a standing backlog**.

- **Worst-Case Well-Behaved Queuing Bound**: When a request from `wb1` arrives at an empty bucket, it is inserted at the back of `active_buckets_` behind at most $N_{\text{active}} - 1 = 3$ other buckets (one turn each for `wb2`, `wb3`, and `noisy`). With 4 upstream connections completing a request every $1/329$ s ≈ 3.04 ms, waiting for 1 to 3 round-robin turns adds only $\sim$ 3–25 ms of queuing delay on top of the 11.8 ms backend service time, yielding the observed **mean latency of** 18.2–21.4 **ms** and **P99 latency of** 44.4–48.7 **ms**.

- **Work Conservation & 51% Queue Depth Reduction**: Because well-behaved clients only consume $3 \times 60 = 180$ RPS, their buckets are empty during the remaining turns, and SFQ work-conservingly dispatches `noisy`'s bucket at $\mu - 180 \approx 149.2$ RPS. Crucially, because the 120 well-behaved requests are no longer trapped waiting behind `noisy`, total Envoy pending queue depth drops from **204.2 requests** (FIFO) to **100.0 requests** (SFQ)—consisting solely of `noisy`'s own backlog! Meanwhile, `noisy` absorbs the full queueing delay of its burst: its tail latency increases from 683.5 ms (P99 under FIFO, where well-behaved clients subsidized its queue position) to 1264.6 ms (P99 under SFQ, since 100 queued requests/149.2 RPS $\approx$ 670 ms mean wait and up to $\sim$ 1.26 s at P99 when all 3 well-behaved buckets interleave).

6.3 Empirical Results Summary Table

Table 4 presents the complete client-observed latency and cluster queue metrics from `tools/queue_policy_test/resul`

Policy	Client	Reqs	Mean (ms)	P50 (ms)	P90 (ms)	P99 (ms)	P99.9 (ms)	Burst RPS	Avg / Max Q
Default	`wb1` (Steady)	1,799	334.44	365.61	666.73	681.54	688.26	57.14	
(`no_queue`)	`wb2` (Steady)	1,799	340.79	436.42	668.70	682.75	690.19	56.99	203.0 avg
	`wb3` (Steady)	1,799	341.05	430.33	670.43	685.93	692.88	57.06	218 max
	`noisy` (Burst)	2,403	602.17	659.91	672.40	684.82	685.77	157.03	
FIFO Ext.	`wb1` (Steady)	1,799	340.98	422.07	673.45	682.75	685.15	57.06	
(`fifo`)	`wb2` (Steady)	1,799	337.98	411.22	670.04	679.84	684.06	57.06	204.2 avg
	`wb3` (Steady)	1,799	336.55	396.21	669.09	678.43	682.43	57.06	217 max
	`noisy` (Burst)	2,409	601.50	660.05	672.60	683.54	685.11	157.94	
Fair Queue	**`wb1` (Steady)**	**1,799**	**21.37**	**20.95**	**36.05**	**48.69**	**56.68**	**59.90**	
(`fair_queue`)	**`wb2` (Steady)**	**1,799**	**19.50**	**17.61**	**34.04**	**45.93**	**53.74**	**59.90**	**100.0 avg**
SFQ	**`wb3` (Steady)**	**1,799**	**18.22**	**15.31**	**31.79**	**44.37**	**50.85**	**59.98**	**104 max**
	`noisy` (Burst)	2,294	630.11	604.01	1092.29	1264.58	1335.10	149.22	
SFQ vs. FIFO	**Well-Behaved**	**0% drop**	−94.2%	−95.6%	−94.9%	−93.2%	−92.1%	+5.0%	−51.0% Q
Improvement	(`wb1..wb3` avg)		(**17.2**×)	(**22.8**×)	(**19.7**×)	(**14.6**×)	(**12.7**×)	(**59.9 RPS**)	

Table 4: End-to-end Nighthawk benchmark results across queue policies (30 s run with $t = 10..25$ s burst).

6.4 Visual Analysis of Benchmark Graphs

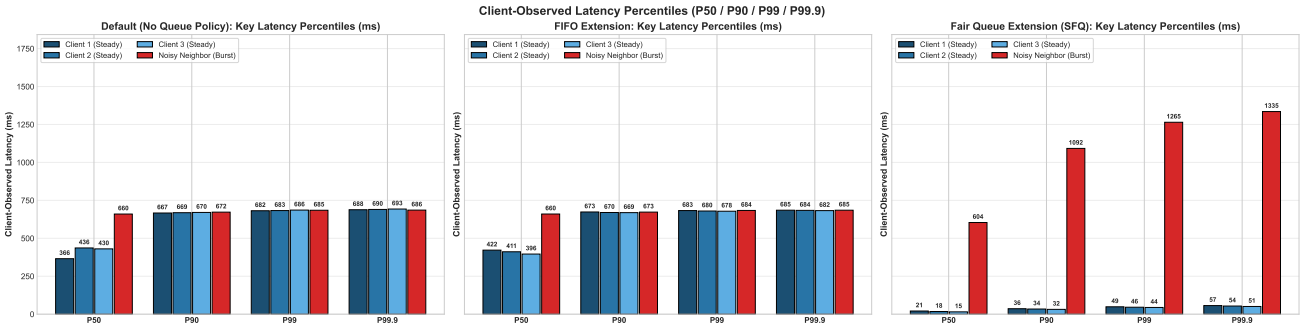


Figure 5: Key client-observed latency percentiles (`P50`, `P90`, `P99`, `P99.9`) across queue policies. Under Default and FIFO, well-behaved clients (`wb1..wb3`, blue bars) suffer the exact same $\sim$ 670–690 ms tail latency as the noisy neighbor (red bar). Under Fair Queue (SFQ), well-behaved tail latency stays below 57 ms even at `P99.9`.

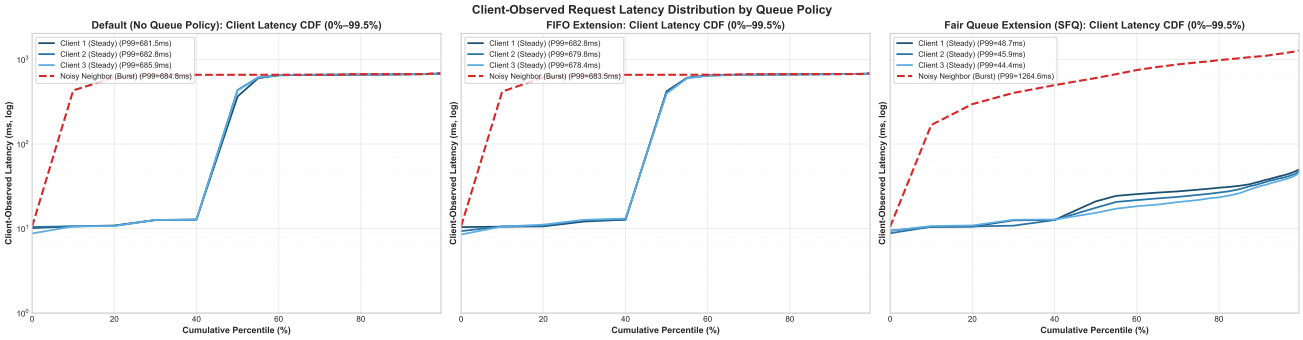


Figure 6: Per-policy Cumulative Distribution Functions (CDFs, log scale, 0%–99.5%). In the Default and FIFO panels, the blue curves (`wb1..wb3`) jump sharply at the 50th percentile (corresponding to the 15 s/30 s = 50% duration of the noisy burst) from 12 ms to 680 ms. In the Fair Queue panel, the blue curves remain flat at 12–48 ms across the entire distribution.

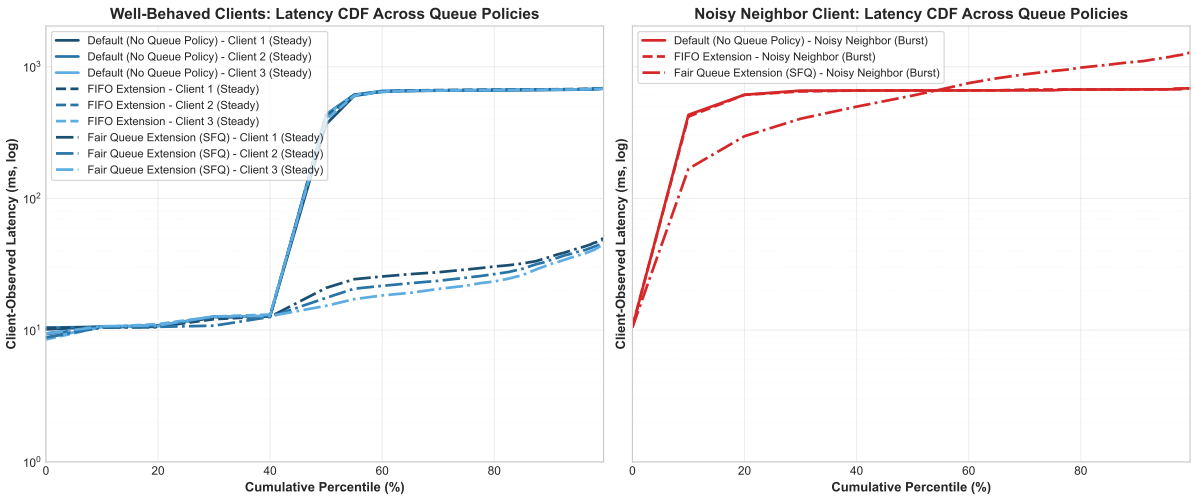


Figure 7: Cross-policy CDF overlay separating Well-Behaved clients (left) from the Noisy Neighbor (right). Left panel shows an order-of-magnitude downward shift in the latency distribution for `wb1..wb3` under SFQ (dash-dot curves). Right panel shows the Noisy Neighbor absorbing its own self-inflicted queuing delay above the 50th percentile.

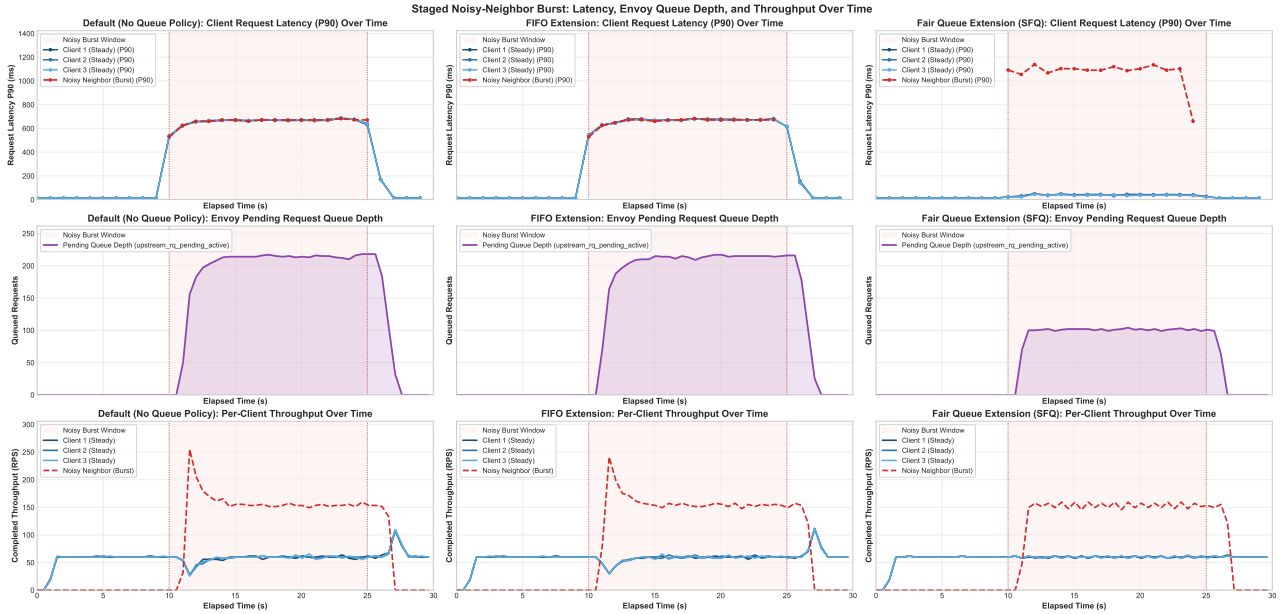


Figure 8: Three-row timeseries across the 30 s run (shaded red region marks the $t = 10..25$ s burst): **Row 1 (P90 Latency)**: Under FIFO, `wb1..wb3` P90 spikes to ~ 670 ms for the entire burst; under SFQ, `wb1..wb3` P90 stays flat at ~ 36–41 ms while `noisy` rises to ~ 1068 ms. **Row 2 (Queue Depth)**: `upstream_rq_pending_active` reaches 217–218 under FIFO vs. 104 under SFQ. **Row 3 (Throughput)**: Under SFQ, `wb1..wb3` maintain a rock-solid 60 RPS throughout the burst.

7 Production Configuration Reference

Listing 6 provides a complete production Envoy configuration pairing `envoy.filters.http.header_to_metadata` on the downstream HTTP Connection Manager with `envoy.queue_policy.fair_queue` on the upstream cluster:

```
static_resources:
  listeners:
  - name: ingress_http_listener
    address:
      socket_address: { address: 0.0.0.0, port_value: 10000 }
    filter_chains:
    - filters:
      - name: envoy.filters.network.http_connection_manager
        typed_config:
          "@type":
              type.googleapis.com/envoy.extensions.filters.network.http_connection_manager.v3.Ht
          stat_prefix: ingress_http
          route_config:
            name: local_route
            virtual_hosts:
            - name: backend_vhost
              domains: ["*"]
              routes:
              - match: { prefix: "/" }
                route: { cluster: backend_cluster }
          http_filters:
          # Step 1: Populate StreamInfo dynamic metadata from the tenant header.
          - name: envoy.filters.http.header_to_metadata
            typed_config:
```

```
              "@type":
                  type.googleapis.com/envoy.extensions.filters.http.header_to_metadata.v3.Config
              request_rules:
              - header: x-client-id
                on_header_present:
                  metadata_namespace: envoy.filters.http.header_to_metadata
                  key: client_id
                  type: STRING
          - name: envoy.filters.http.router
            typed_config:
              "@type":
                  type.googleapis.com/envoy.extensions.filters.http.router.v3.Router

  clusters:
  - name: backend_cluster
    type: STRICT_DNS
    connect_timeout: 5s
    lb_policy: ROUND_ROBIN
    circuit_breakers:
      thresholds:
      - priority: DEFAULT
        max_connections: 64
        max_pending_requests: 4096
        max_requests: 4096
    # Step 2: Configure Stochastic Fair Queuing (SFQ) on the cluster pending stream
        queue.
    queuing_policies:
      pending_rq_policy:
        name: envoy.queue_policy.fair_queue
        typed_config:
          "@type":
              type.googleapis.com/envoy.extensions.queue_policy.fair_queue.v3.FairQueuePolicyCon
          class_metadata_key:
            key: envoy.filters.http.header_to_metadata
            path:
            - key: client_id
          num_buckets: 1024
          perturbation_interval: 1s
```

Listing 6: Complete Envoy configuration for tenant-isolated upstream pending request queuing.

7.1 Key Source Files Reference

- **Cluster API**: `api/envoy/config/cluster/v3/cluster.proto` (`Cluster.QueuingPolicies`)
- **Queue Policy APIs**: `api/envoy/extensions/queue_policy/fifo/v3/fifo.proto`, `api/envoy/extensions`
- **Core Interface & Default FIFO**: `source/common/queue_policy/queue_policy_base.h`, `source/common/q`
- **Fair Queue (SFQ) Implementation**: `source/extensions/queue_policy/fair_queue/fair_queue_policy` `fair_queue_policy.cc`
- **Connection Pool & Router Plumbing**: `source/common/conn_pool/conn_pool_base.h/.cc`, `source/common/http/conn_pool_base.h/.cc`, `source/common/router/upstream_request.cc`, `source/common/upstream/upstream_impl.cc`
- **Benchmark Suite & Plots**: `tools/queue_policy_test/scenario.yaml`, `run_benchmark.py`, `generate_plots.py`, `results/`